

I - Annexe – énoncés de TP

1. Bibliographie et liens vers sites "internet"

https://git-scm.com/	site officiel de la technologie git
https://openclassrooms.com/courses/gerez-vos-codes-source-avec-git	tutoriel sur GIT

2. TP

2.1. Installation de git en mode texte

Installer git en fonction du type d'ordinateur

Exemple pour linux debian ou ubuntu :

```
$ sudo apt update
$ sudo apt upgrade -y
$ sudo apt install git
$ git --version
```

Exemple pour windows :

<https://git-scm.com/download> site officiel pour télécharger l'installateur de git

<https://github.com/git-for-windows/git/releases/download/v2.43.0.windows.1/Git-2.43.0-64-bit.exe>

Lancer l'installateur avec choix par défauts

Vérification:

```
git --version
git version 2.43.0.windows.1
```

2.2. Configuration locale fondamentale de git

```
git config --global user.name "PrenomNom"
git config --global user.email "prenom.nom@ici\_ou\_la.fr"
git config --global init.defaultbranch main
git config --show-scope --list
```

2.3. Eventuelle installation de compléments graphiques :

Liste des clients "git" graphiques: <https://git-scm.com/downloads/guis>

TortoiseGit (*gratuit , sous windows*)

<https://tortoisegit.org/download/>

<https://download.tortoisegit.org/tgit/2.14.0.0/TortoiseGit-2.14.0.1-64bit.msi>

GitKraken

<https://www.gitkraken.com/download/windows64> (*produit payant , version gratuite limitée à 7jours à priori*)

SmartGit

+ *plugins "git" des principaux IDE (eclipse, intelliJ , Visual studio code , ...)*

2.4. Git élémentaire en mode local

Régler si besoin l'explorateur de fichier windows pour qu'il affiche les extensions des noms de fichiers (ex : *.txt*) et qu'il affiche les fichiers et répertoires cachés.

Se créer un répertoire de tp (ex: *c:\tp\tp-git*)

Créer les sous répertoires *c:\tp\tp-git\local-git-repositories-devA*

et *c:\tp\tp-git\local-git-repositories-devA\p1*

Au sein du répertoire *p1* , créer (par exemple via notepad++) les fichiers simples suivants :

p1\fl.txt

```
ligne1
ligne2
```

p1\f2.txt

```
abc
def
```

p1\inutile.bad

```
fichier pas toujours utile et potentiellement de grande taille
```

Création d'un référentiel local git au sein du projet *p1* :

se placer dans *local-git-repositories-devA\p1*

```
git init
```

→ Visualiser via l'explorateur de fichiers , le **nouveau sous répertoire caché ".git"** .

Générer (dans *p1*) le souvent indispensable fichier **.gitignore**

```
echo *.bad > .gitignore
```

Visualiser le contenu de **.gitignore** via notepad++ et ajouter y les lignes suivantes :

.gitignore

```
*.bad
*.jar
/node_modules
/bin
/target
```

Lancer les commandes suivantes et visualiser bien les effets de chacune d'elles

```
git status
git add *.txt
git add .gitignore
git status
git commit -m "initial"
git status
git log
```

En exercice :

- **ajouter** le fichier *f3.txt* contenant "*f3 c'est 3 fois rien*"
- **modifier** le contenu de *f1.txt* avec quelques majuscules
- lancer **git status** et **git diff** pour afficher les modifications pas encore enregistrées
- lancer **git status** , **git add ...** , **git commit -a -m "..."** de façon à enregistrer tous les changements
- réafficher l'historique via **git log**
- **ajouter** le fichier *f4.txt* contenant "*f4 est un nouveau fichier*"
- **modifier** le contenu de *f1.txt* avec une ligne en plus
- lancer **git status** , **git add ...** , **git commit -a -m "..."** de façon à enregistrer tous les changements
- **réafficher l'historique via git log**
et copier dans le presse papier l'**id de l'avant dernier commit** associé à "*f3 et modif f1*"
(exemple : *775c58b7a5f39c2d2d61d1e0bcc7bcbe3f412905*)

Retour en arrière (vers ancienne version sauvegardée) :

```
git checkout 775c58b7a5f39c2d2d61d1e0bcc7bcbe3f412905
```

→ visualiser le nouvel état de p1 (f4.txt a disparu et f1.txt ne comporte plus le 3ème ligne ajoutée)

Revenir en tête de branche principale :

```
git checkout main (ou bien git checkout master)
git log
```

→ visualiser le nouvel état de p1 (f4.txt est revenu et f1.txt comporte la 3ème ligne ajoutée)

Associer le tag v1 à l'avant dernier commit :

```
git tag v1 775c58b7a5f39c2d2d61d1e0bcc7bcbe3f412905
git tag -l
git log
gitk
```

git checkout tags/v1

→ visualiser l'absence de f4.txt

git checkout main

→ visualiser le retour de f4.txt

2.5. Quelques expérimentations facultatives conseillées

- Après un commit et un message de type "nothing to commit, working tree clean"
ajouter un nouveau fichier **f_erreur.txt** contenant "*mauvaise idée*", l'ajouter dans l'index via **git add ...** . Lancer **git status** , se dire finalement qu'il faut annuler le "git add" en lançant **git reset f_erreur.txt** Vérifier l'annulation via git status . Supprimer le fichier f_erreur.txt
- En étant positionné en tête de branche principale (main ou master), modifier le fichier fl.txt en ajoutant une nouvelle dernière ligne de type "*mauvaise idée*" .
Lancer la commande **git restore fl.txt** pour restaurer le fichier fl.txt dans l'état du dernier commit enregistré. Visualiser la disparition de "*mauvaise idée*" .
- Expérimenter **git show** sur **fl.txt** pour afficher une ancienne version du fichier fl.txt sans effectuer de checkout (conseil : git log préalable pour identifier une ancienne version).
- Expérimenter l'option --amend de git commit en :
 - affichant une première fois l'historique via **git log -n 4**
 - ajoutant le nouveau fichier ficAa.txt et un git add ... puis un git commit via l'option --amend -m "ajout ficAa.txt"
 - ajoutant le nouveau fichier ficBb.txt et un git add ... puis un git commit via l'option --amend -m "ajout ficAa.txt et ficBb.txt"
 - réafficher une nouvelle fois l'historique via **git log -n 4**
- Expérimenter quelques filtrages sur git log (par contenu de message par exemple)

2.6. Gestion élémentaire des branches de git

git branch

→ par défaut une seule branche master ou **main**

création d'une nouvelle branche dev :

git branch dev

git branch

Se positionner sur branche dev

git checkout dev

git branch

main (ou master)

** dev (avec * devant branche courante/sélectionnée).*

En exercice :

- ajout de f5.txt et nouveau commit sur branche dev
- ajout de f6.txt et encore nouveau commit sur branche dev
- git log
- se positionner sur branche main ou master (via git checkout)
→ visualiser absence de f5.txt et f6.txt
- se positionner sur branche dev (via git checkout)
→ visualiser retour de f5.txt et f6.txt

Fusion élémentaire de branches (fast forward) :

Modifications effectuées que sur dev , aucune modif effectuées sur main → fast forward

git checkout main (ou bien *git checkout master*)

git merge dev

git log

NB : ne pas supprimer la branche dev (qui sera utilisée au sein des prochains TPs)

2.7. Merge de fusion sans conflit

En exercice :

- partir du sommet de la branche dev
- créer une nouvelle branche temporaire fb2
- se placer sur la branche fb2
- ajouter le fichier fa.txt
- **ajouter *** en fin de la première ligne de f1.txt**
- commiter tous ces changements (sur branche actuelle fb2)
- se placer sur la branche dev
- ajouter fb.txt
- ajouter *dernièreNouvelleligne###* en **fin de f1.txt** (sans changer les premières lignes)
- commiter tous ces changements (sur branche actuelle dev)
- rester sur la branche dev
- déclencher une fusion avec la branche fb2
- visualiser tous les effets via `git log -n 6 --oneline --all --graph`
(nouveau commit de fusion, état des fichiers fa.txt , fb.txt , f1.txt)
- supprimer l'ancienne branche temporaire fb2

2.8. Merge git avec résolution de conflit

- partir du sommet de la branche dev
- créer une nouvelle branche temporaire fb3
- se placer sur la branche fb3
- modifier la première ligne de f2.txt (abc transformé en ABC)
- commiter tous ces changements (sur branche actuelle fb3)
- se placer sur la branche dev
- modifier différemment la première ligne de f2.txt (abc transformé en a_b_c)
- commiter tous ces changements (sur branche actuelle dev)
- rester sur la branche dev
- déclencher une fusion avec la branche fb3
- visualiser tous le conflit (message de la commande, état du fichier f2.txt)

Auto-merging f2.txt

CONFLICT (content): Merge conflict in f2.txt

Automatic merge failed; fix conflicts and then commit the result.

- **git status**

- On pourrait éventuellement annuler la fusion via `git merge --abort` (si trop de conflits à résoudre). On va préférer résoudre le conflit pour finaliser la fusion.
- visualiser l'état temporaire de **f2.txt**

```
<<<<<<< HEAD
a_b_c
=====
ABC
>>>>>>> fb3
def
```

- ne garder dans **f2.txt** qu'une des versions proposées a_b_c ou ABC ou un mixte des 2.

```
A_B_C
def
```

- indiquer la fin de la résolution du conflit sur f2.txt via la commande **git add f2.txt**
- finaliser la fusion via la commande **git commit -m "merge branch fb3"**
- `git log --oneline --all --graph`
- supprimer l'ancienne branche temporaire fb3

NB : on pourra facultativement expérimenter un merge avec un IDE tel que eclipse, intelliJ ou VisualStudioCode .

2.9. Git élémentaire en mode remote (clone, push, pull)

Se placer dans répertoire de tp (ex: `c:\tp\tp-git`)

Créer les sous répertoires `c:\tp\tp-git\local-git-repositories-devB`

et `c:\tp\tp-git\git-shared-repositories`

NB :

- si le début du chemin "`c:\tp\tp-git`" est différent sur votre PC , il faudra alors adapter ce chemin dans toute la suite de l'énoncé de ce TP .
- `git-shared-repositories` regroupera des référentiels git partagés (éventuellement accessibles à distance selon accès réseaux ...)
- `local-git-repositories-devA` regroupera des référentiels locaux pour le développeur `devA`
- `local-git-repositories-devB` regroupera des référentiels locaux pour le développeur `devB`

Se placer dans le répertoire `c:\tp\tp-git\git-shared-repositories`

et créer le sous répertoire `c:\tp\tp-git\git-shared-repositories\p1.git`

Se placer dans le répertoire `c:\tp\tp-git\git-shared-repositories\p1.git`

et initialiser le référentiel distant/partagé en lançant la commande

```
git init --bare
```

Visualiser le contenu de `p1.git`

Référencement de `p1.git` en tant que version distante/partagée de `local-git-repositories-devA/p1`

Se placer dans `local-git-repositories-devA/p1` et lancer les commandes suivantes :

```
git remote -v
```

(`git remote remove origin` pour supprimer ancienne config en cas d'erreur d'URL)

```
git remote add origin file:///C:/tp/tp-git/git-shared-repositories/p1.git
```

```
git remote -v
```

NB: `origin` est à voir comme un `alias` pour l'url `file:///C:/tp/tp-git/git-shared-repositories/p1.git`

Si `git_bash/mingw64` alors url à ajuster de type `file:///c:/tp/tp-git/git-shared-repositories/p1.git`

Première publication partagée depuis `local-git-repositories-devA/p1`:

```
git push --set-upstream origin main
```

Clonage du référentiel partagé depuis `local-git-repositories-devB` (développeur B) :

Se placer dans `c:\tp\tp-git\local-git-repositories-devB`

git clone file:///C:\tp\tp-git\git-shared-repositories\p1.git
ou bien <file:///c:/tp/tp-git/git-shared-repositories/p1.git>

→ Visualiser le sous répertoire p1 construit (par clonage)

cd p1 (depuis ...-devB)

git remote -v

origin file:///C:\tp\tp-git\git-shared-repositories\p1.git (fetch)

origin file:///C:\tp\tp-git\git-shared-repositories\p1.git (push)

git checkout main

→ visualiser les copies locales des fichiers fl.txt , ...

Modif et publication depuis un développeur de l'équipe dev-B ou dev-A

- modifier un des fichiers (ex : nouvelle ligne dans fl.txt)
- intégration d'éventuelles modifications distantes via git pull
- commit local
- intégration d'éventuelles modifications distantes via git pull
- publication via **git push**

Rapatriement des modifications enregistrées/partagées depuis autre développeur dev-B ou dev-A

- **git pull**
- visualiser les changements

Variantes d'URL pour ce TP :

<file:///C:/tp/tp-git/git-shared-repositories/p1.git> (sur un seul ordinateur)

<file:///Y:/git-shared-repositories/p1.git> où Y: est un lecteur réseau associé à un répertoire partagé par un autre ordinateur via le réseau

<https://github.com/xyz/p1.git> où p1.git est un référentiel git partagé géré par l'utilisateur xyz ayant un compte sur [github](https://github.com) .

2.10. Merge git avec résolution de conflit et branches distantes

Expérimenter des commits/push/pull avec conflits à résoudre entre deux utilisateurs travaillant sur un même fichier d'un même projet (ex : p1).

On pourra effectuer se Tp en simulant deux utilisateurs sur le même ordinateur (configuration du TP précédent) ou bien en configurant plusieurs comptes **github** ou **gitlab** ou **gitbucket** .

Bonnes pratiques (en général) :

- d'abord un commit sur branche locale
- pull avant merge ou push

2.11. publication/partage de branche

- Du côté dev-A , dans p1 , lancer la commande nécessaire (de type *git push -u origin ...*) pour partager/publier la branche dev
- Du côté dev-B , dans p1 , mettre à jour certaines méta-données via git fetch puis lancer la commande *git branch -a* pour visualiser l'existence de origin/dev
- du côté dev-B, lancer la commande suivante pour créer une version locale de dev :
git checkout -b dev origin/dev
- Vérifier via **git branch -vv**

2.12. accès distant à un référentiel github ou gitlab

Selon vos préférences (github ou gitlab), on pourra effectuer ce TP dans (entre autres) l'une des deux versions présentées ci-après (a : avec github , b : avec gitlab) avant d'effectuer partie commune (c: manipulations sur le clone git-classroom) .

2.12.a. accès github via token

<https://github.com/didier-tp/git-classroom.git> est l'URL d'un référentiel distant préparé pour quelques Tps .

Ce référentiel (de type "sandbox" sans rien d'important à l'intérieur) appartient à l'utilisateur didier-tp et l'on peut tout de même y accéder en lecture écriture via un jeton d'accès personnel .

Token (révocable) permettant d'accéder qu'à ce référentiel de Tp :

```
github_pat_11AHMRVJQ0WZrJ1X5kYcdm_GohIS6u6uNisLW4ESk399AI5Q6ZNVqa7WwRIC2JnQxFCDNVHCQ6HejdgfV
```

URL adaptée avec le token (*origin* à changer après clonage):

https://didier-tp:the_token_value@github.com/didier-tp/git-classroom.git

```
https://didier-tp:github_pat_11AHMRVJQ0WZrJ1X5kYcdm_GohIS6u6uNisLW4ESk399AI5Q6ZNVqa7WwRIC2JnQxFCDNVHCQ6HejdgfV@github.com/didier-tp/git-classroom.git
```

NB: Si, à la date du TP, le jeton valide a changé de valeur, on pourra selon les consignes du formateur, créer un fichier temporaire **url.txt** pour y mettre au point (par concaténation adaptée/ajustée) l'url complète avec le nouveau jeton d'accès valide.

1. Se placer dans le répertoire c:\tp\tp-git\local-git-repositories-devA (ou un équivalent)
2. Créer un clone du référentiel <https://github.com/didier-tp/git-classroom.git> avec idéalement l'url complétée avec le jeton d'accès.
3. Seulement si l'URL comportant le jeton est erronée et quelle a besoin d'être mise à jour , on pourra alors lancer la commande suivante (à adapter avec un jeton valide) :
git remote set-url origin https://didier-tp:bonne_valeur_jeton@github.com/didier-tp/git-classroom.git

2.12.b. accès distant gitlab avec clef ssh

1. visualiser <https://gitlab.com/didier-lab/git-classroom> avec un navigateur
2. dans C:\tp\tp-git\local-git-repositories-devA créer un clone de <https://gitlab.com/didier-lab/git-classroom>
3. tenter un ajout de type monPrenom.txt , commit et push : le push sera refusé sans mot de passe (que didier ne divulguera pas)
4. dans C:\Users\Administrateur (ou ailleurs : votre \$HOME) créer si besoin un sous répertoire .ssh
5. recopier C:\tp\tp-git\local-git-repositories-devA\git-classroom\temp\id_rsa_old dans C:\Users\Administrateur\.ssh\id_rsa (sans _old par renommage de la copie).
et C:\tp\tp-git\local-git-repositories-devA\git-classroom\temp\id_rsa_.pub dans C:\Users\Administrateur\.ssh\id_rsa.pub
6. changer l'url avec **git remote set-url origin** [git@gitlab.com:didier-lab/git-classroom.git](https://gitlab.com/didier-lab/git-classroom.git)
7. retenter git push avec si besoin yes à la question posée
et si les clefs ssh ne fonctionnent pas bien on pourra si besoin demander à en générer de nouvelles...

2.12.c. manipulations au sein du clone git-classroom

Se placer dans le répertoire ...**git-classroom**

Lancer ensuite un enchaînement de modifications et de commandes selon cette logique :

- **git branch -a** (pour voir toutes les branches "remotes")
- **git checkout -b m-dev origin/m-dev** (créer branche locale "m-dev" pistant la branche distante origin/m-dev)
- **git branch -vv** (affichage avec détails sur pistages)
- Depuis un éditeur quelconque (notepad++ , ...) ou bien un IDE quelconque (eclipse ou intelliJ ou VisualStudioCode ou ...), modifier le contenu du fichier **notes.txt** (à peut être créer s'il n'existe pas encore)
essais_en_solo/s1_ou_..._ou_s15/notes.txt
- répercuter la mise à jour au niveau du référentiel partagé (add,commit,push, ...)
- visualiser le contenu partagé de **essais_en_solo/s1_ou_..._ou_s15/notes.txt** sur la branche **m-dev** du référentiel sur le site de github via un navigateur internet (<https://github.com/didier-tp/git-classroom>)

Suite du tp (selon le contexte de l'environnement de TP) :

- Si "suite du Tp réalisée seul" , alors refaire toutes les manipulations précédentes au sein d'un sous répertoire "**local-git-repositories-devB**" de **C:\tp\tp-git**.
- Si "suite du Tp réalisée en groupe" , alors "chaque stagiaire" dispose d'un clone sur son poste dans un répertoire **C:\tp\tp-git\local-git-repositories-devA\git-classroom** (ou un équivalent).

Suite du Tp (travail en pseudo-équipe réelle ou simulée) :

- effectuer des "**séquences modif/add/commit/push**" de tous les cotés (devA, devB , ...) de différentes manières :
 - avec concertation et pull préalable (chacun son tour en attendant l'autre)
 - sans pull préalable
 - sans concertation au début et concertation lors des "merges_avec_conflits"

NB : on pourra par exemple effectuer des modifications sur le fichier

essais_en_groupes/g1_ou_..._ou_g10/notes.txt (à créer si pas encore existant)
ou bien **essais_en_solo/s1_ou_..._ou_s15/notes.txt**

ou bien ajouter un nouveau fichier

essais_en_groupes/g1_ou_..._ou_g10/votre_prenom.txt ou bien **essais_en_solo/s1_ou_..._ou_s15/votre_prenom.txt**

2.13. Avec son propre compte github ou gitlab (tp facultatif)

1. Se créer un compte gratuit sur github ou gitlab (une adresse email devra être vérifiée) et mémoriser les infos d'authentification (username/password ou token ou ...)
2. Créer un nouveau référentiel distant (vide au départ)
3. Publier un projet local vers le référentiel distant (git remote ... , git push ...)
4. Cloner le référentiel distant ailleurs (git clone ,)
5. effectuer une série de changements (commit , push , pull, ...) sans conflit soit en ligne de commande , soit via tortoiseGit , soit via un IDE (ex : eclipse).
6. On pourra éventuellement autoriser d'autres développeurs amis à effectuer des publications sur notre référentiel partagé github ou gitlab .
7. On pourra expérimenter un "merge request" gitlab ou bien "pull request" github
A défaut, une petite "démo" pourra être demandée au formateur.

2.14. Git depuis un IDE (Tp facultatif)

1. Vérifier la présence (dans C:\tp\tp-git\local-git-repositories-devA ou ailleurs) d'un clone de **git-classroom** et si besoin créer ce clone selon un des Tps précédents .
2. Charger un des sous projets (*basicHtmlCssJs* ou *basicJava* ou *basicNodeJs*) depuis un IDE (ex : eclipse , intelliJ , visual studio code, ...)
3. Expérimenter les opérations git basiques (commit, push, pull)
4. Expérimenter éventuellement les résolutions de conflits entre branches locales depuis l'IDE

2.15. Expérimentation de "rebase" sur branches locales

On pourra par exemple réutiliser le référentiel p1.git
et l'un de ses clones (ex : *C:\tp\tp-git\local-git-repositories-devA\p1*)

- partir de la branche "**dev**"
- créer une nouvelle branche **fbt** et se placer dessus
- effectuer deux ajouts/commits sur la branche **fbt**
- revenir sur la branche "**dev**"
- effectuer deux ajouts/commits différents (sans conflits) sur la branche "**dev**"
- se placer sur la branche **fbt**
- afficher l'historique de **fbt** via la commande **git log --oneline --all --graph**
- *rebase la branche courante **fbt** sur la tête de la branche **dev** :*
git rebase dev
- réafficher le nouvel historique de *fbt* (changé) via la commande **git log --oneline --all --graph**
- se replacer sur la branche dev
- afficher l'historique de *dev* via la commande **git log --oneline --all --graph**
- faire avancer la branche *dev* vers la fin de la branche *fbt* rebasée
de manière à absorber les changements via un merge de type fast-forward :
git merge fbt

- réafficher l'historique de main via la commande `git log --oneline --all --graph`
- **supprimer** la branche `fbt`

2.16. Expérimentation de "cherry-pick" sur branches locales

On pourra par exemple réutiliser le référentiel `p1.git` et l'un de ses clones (ex : `C:\tp\tp-git\local-git-repositories-devA\p1`)

- Se placer sur la branche "dev" (issue de "main")
- effectuer un ajout/commit sur la branche dev et afficher l'historique
- revenir sur la branche "main"
- effectuer l'ajout d'un fichier `myhotfix.txt` comportant une ligne de type MYHOTFIX. et un commit ayant pour message "myhotfix" sur la branche "main"
- copier dans le presse-papier l'identifiant au format "sha1" du commit "myhotfix" (exemple `dc3ae11030331cb69d7463d7a0a2c0a6153a9447`) que l'on peut trouver via une commande comme `git log -n 3`
- se replacer sur la branche `dev`
- `git cherry-pick <shaDuCommitMyHotfix>`
(exemple: `git cherry-pick dc3ae11030331cb69d7463d7a0a2c0a6153a9447`)
- réafficher le nouvel historique de dev (changé) via la commande `git log --oneline`
- maintenant que le hotfix (correction de bug d'urgence sur "main") a été rapatrié sur la branche dev, on peut tranquillement continuer à travailler sur la branche dev avec une bonne prise en compte de ce correctif .

2.17. Expérimentation de "stash"

On pourra par exemple réutiliser le référentiel `p1.git` et l'un de ses clones (ex : `C:\tp\tp-git\local-git-repositories-devA\p1`)

- partir de la branche "`dev`"
- effectuer si besoin un `git push` et/ou `git pull`
- créer une nouvelle branche "`fbf`" et rester sur la branche "dev".
- Ajouter un fichier `ficzz.txt` (avec contenu `mm_zz`) et modifier un fichier existant (ex : `f2.txt`) sur la branche dev et commiter cela
- se placer sur la branche "`fbf`" créée précédemment.
- ajouter un nouveau fichier de nom `ficzz.txt` (avec contenu différent : `bb_zz`) sans commiter ce changement
- tenter de revenir sur la branche "dev" (message d'erreur et on est toujours sur branche fbf , à visualiser via "git branch")
- `git stash -u`
- **visualiser la disparition du fichier `ficzz.txt` (sur branche `fbf`) dans le répertoire de travail**
- revenir sur la branche "`dev`" et ré-effectuer un petit changement sur `f2.txt` (à commiter)
- effectuer un "`git push`" sur la branche "dev"
- revenir sur la branche "`fbf`" (où le fichier `ficzz.txt` n'apparaît toujours pas)
- `git stash list`

- **git stash pop**
- visualiser la **réapparition** du fichier *ficzz.txt* dans le répertoire de travail
- commiter l'ajout du fichier *ficzz.txt*
- revenir sur la branche "dev" , fusionner "fbb" et détruire "fbb".
- **git push** et **git log**

2.18. Autres manipulations de branches et tags

On pourra par exemple réutiliser le référentiel p1.git et l'un de ses clones (ex : *C:\tp\tp-git\local-git-repositories-devA\p1*)

- se placer en tête de la branche "dev"
- créer une nouvelle branche locale "fcc" et se placer dessus
- coder un nouveau fichier *xyz.txt* (contenu xyz) et effectuer un add/commit
- coder un fichier *bidon.txt* (contenu bidon) et effectuer un add/commit
- coder un fichier *bidon2.txt* (contenu bidon2) et effectuer un add/commit
- afficher l'historique des 5 derniers commits
- annuler les 2 derniers commits via la commande **git reset --hard HEAD~2**
- Ré-afficher l'historique des 5 derniers commits et visualiser la disparition des fichiers *bidon.txt* et *bidon2.txt*
- *NB : tant que l'on travaille en local , il est même possible d'annuler des commits de fusions/merges .*
- publier/partager la branche fcc vers origin en utilisant l'option -u de git push.
- Du côté **local-git-repositories-devB\p1** , récupérer une copie de la branche **origin/fcc** et visualiser le fichier *xyz.txt*
- Du côté **local-git-repositories-devA\p1** :
 - fusionner fcc sur dev (se placer sur dev et git merge fcc)
 - détruire les versions "remote" et "locale" de fcc via **git push origin --delete fcc** et **git branch -d fcc**
- Du côté **local-git-repositories-devB\p1** :
 - se replacer sur la branche dev
 - lancer la commande **git fetch --prune** et visualiser via **git branch -vv**

2.19. Autres expérimentations secondaires libres

- git squash (en mode graphique via tortoise-git ou bien en mode texte)
- git patch
- ...